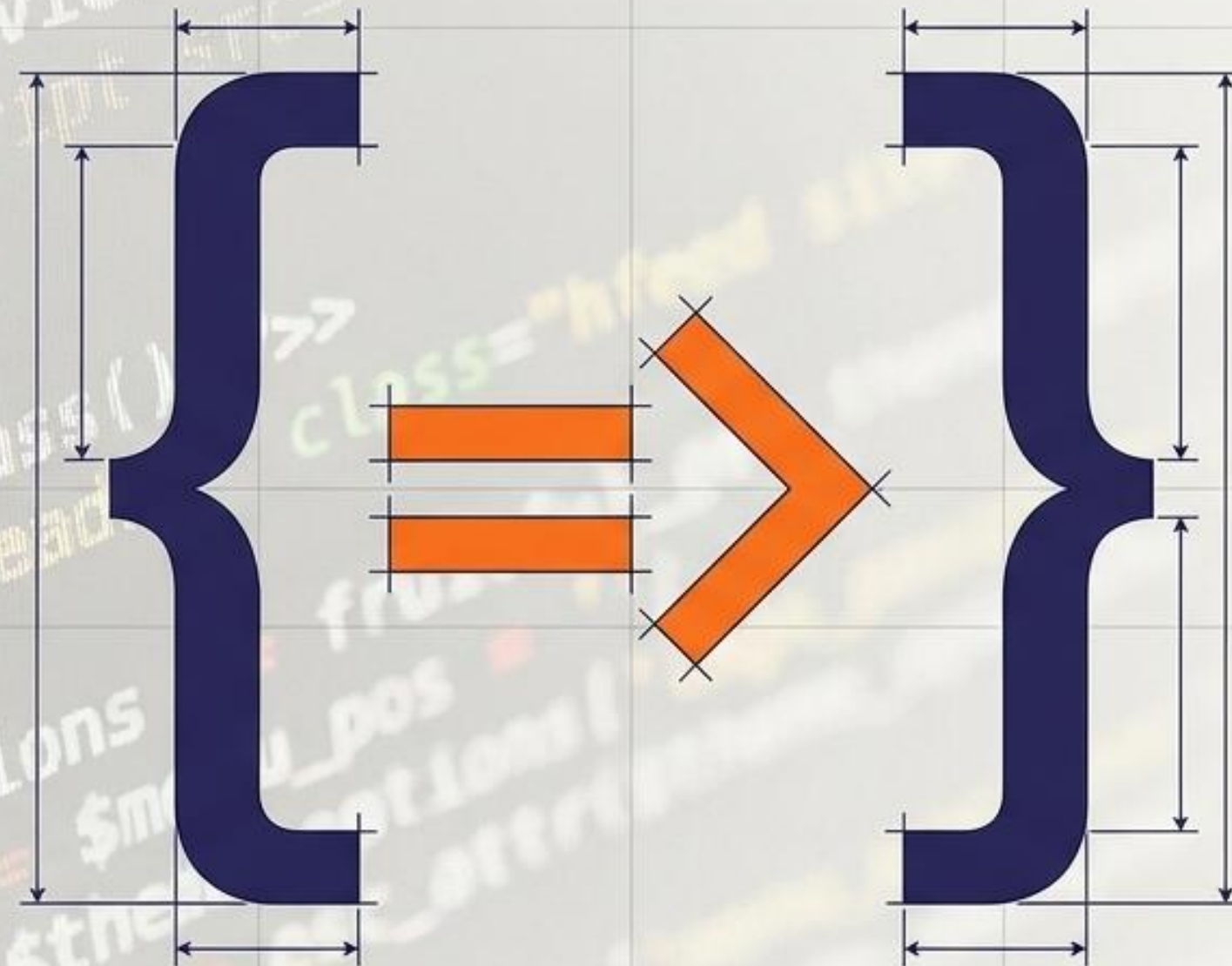




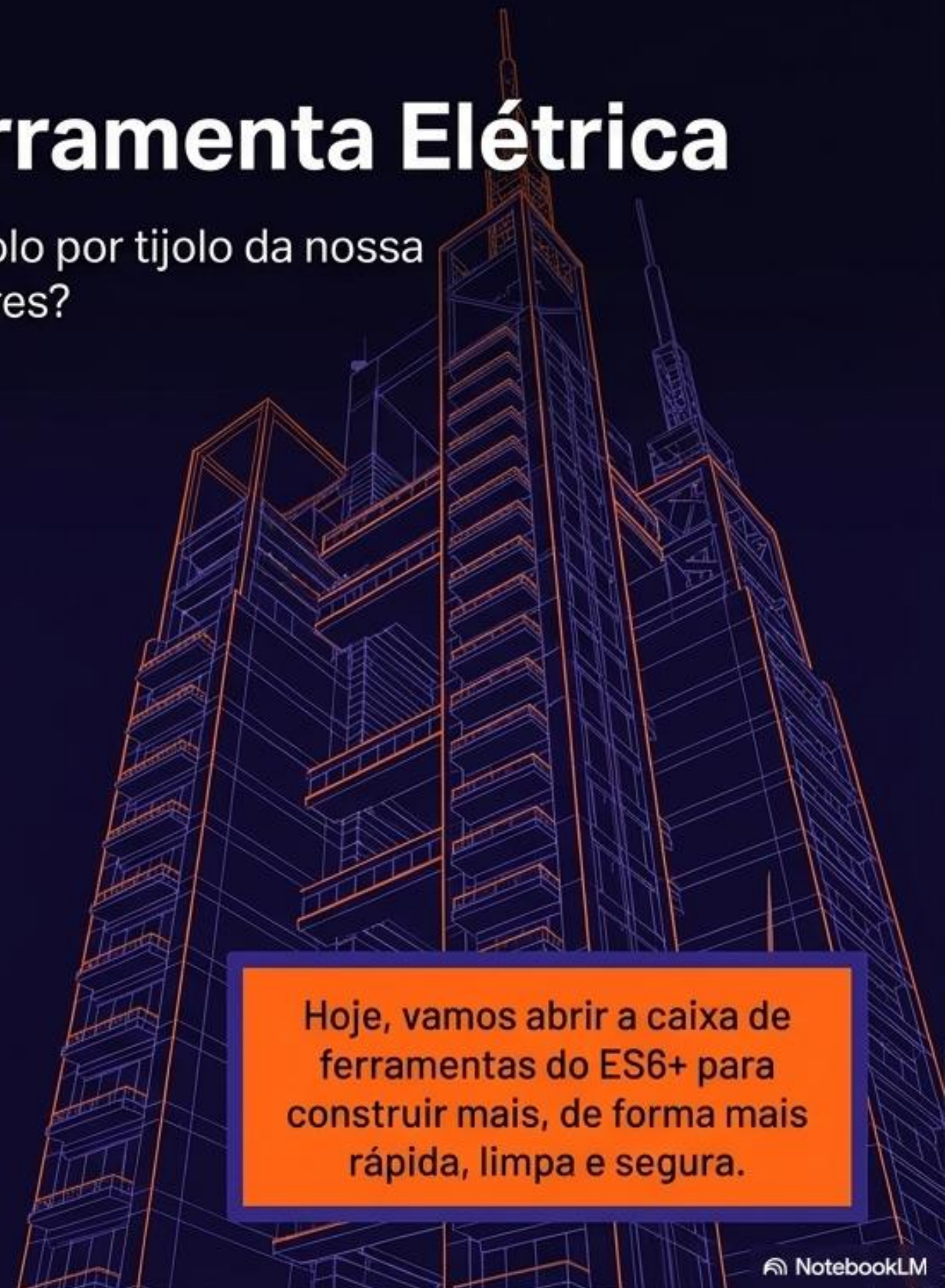
JavaScript Moderno (ES6+) e Listas Dinâmicas

Dando vida aos dados na web.

Programação Front-end 2 (PFE2) – Aula 02

Da Ferramenta Manual à Ferramenta Elétrica

Na última aula, usamos ferramentas manuais para manipular tijolo por tijolo da nossa página. Mas e se precisarmos construir um prédio de 100 andares?



Hoje, vamos abrir a caixa de ferramentas do ES6+ para construir mais, de forma mais rápida, limpa e segura.

O Mapa de Execução de Hoje



O Problema do Escopo Global

O antigo comando `var` é como um post-it solto no meio do escritório: qualquer um pode ler, qualquer um pode alterar, e muitas vezes ele se perde ou gera conflitos.

Precisávamos de previsibilidade.



A Era de Ouro do 'var' Acabou.

O Novo Padrão de Variáveis

Característica	var (Evitar)	 let (Usar quando mudar)	 const (Preferir sempre)
Uso	Sim	Sim	Não
Reatribuição	Alto	Sim	Não
Risco de Sobrescrita	Alto	Baixo	Zero

Regra de ouro: Use const sempre. Só mude para let se tiver certeza absoluta de que o valor precisa mudar durante a execução.

Arrow Functions: Ordens Diretas

É uma **forma mais direta** de dar uma ordem. Em vez de "Por favor, execute a seguinte função...", é simplesmente "**Faça isto =>**".

```
function(a, b) { return a + b }
```



```
const somar = (a, b) => a + b;
```


Manipulação de Strings sem Dor

Antes (Com Dor)



Agora (Sem Dor)

``${Olá, nome!}``

 **Múltiplas Linhas:** Permite strings em várias linhas sem quebrar o código (usando crases `).

 **Interpolação:** Insere variáveis diretamente na string usando `${variavel}`.

 **Legibilidade:** Torna o código muito mais limpo para ler e manter.

Laboratório 1: Limpando a Casa (Guiado)

Antes (Com Dor)

1. Onde podemos trocar let/var por const?

Dica: Variáveis que selecionam elementos do DOM raramente mudam de valor.

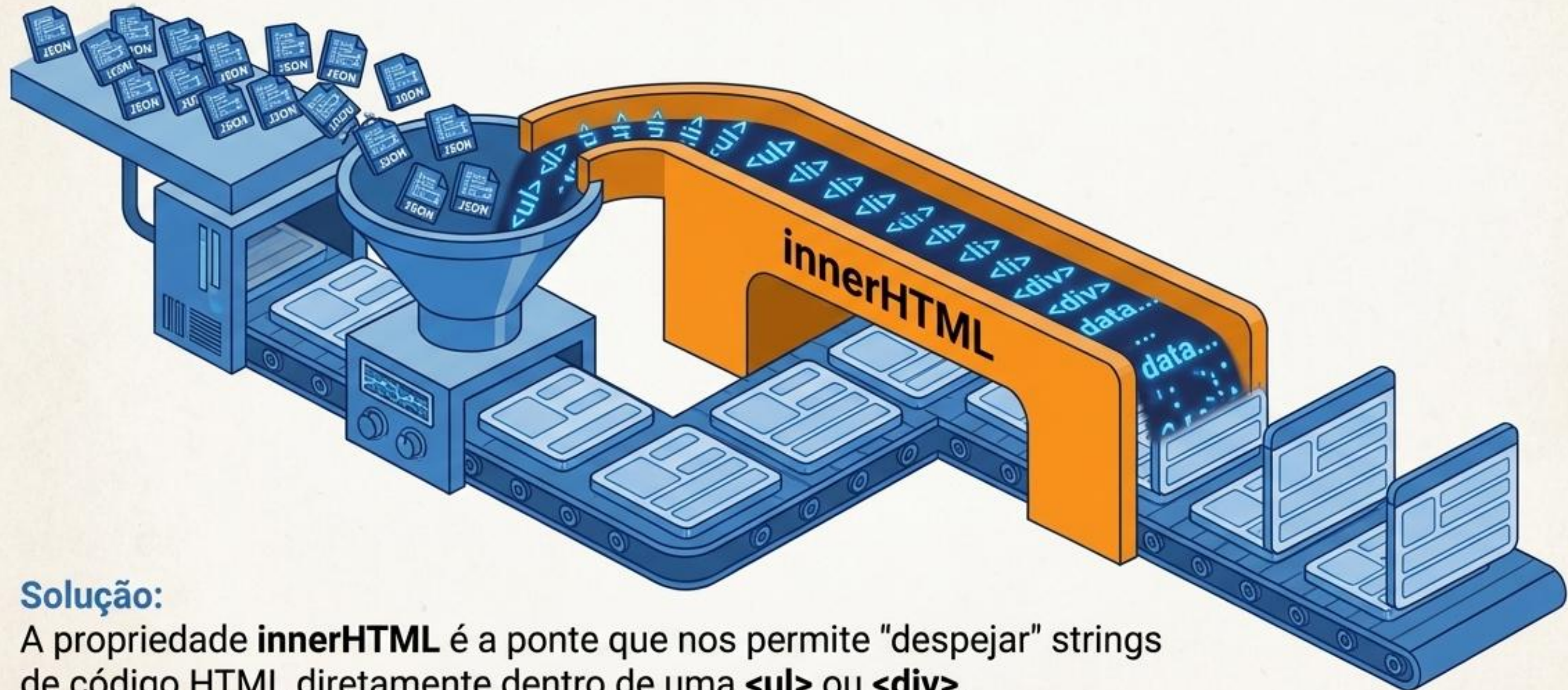
2. Como reescrevemos os callbacks?

Agora (Sem Dor)

Transforme todos os **addEventListener** para utilizarem Arrow Functions de forma concisa.

O Desafio de Renderizar Listas

Recebemos **milhares** de dados de uma API, mas eles estão no formato '**JavaScript**'. O navegador só entende HTML. **Como exibir 1.000 itens na tela em milissegundos?**

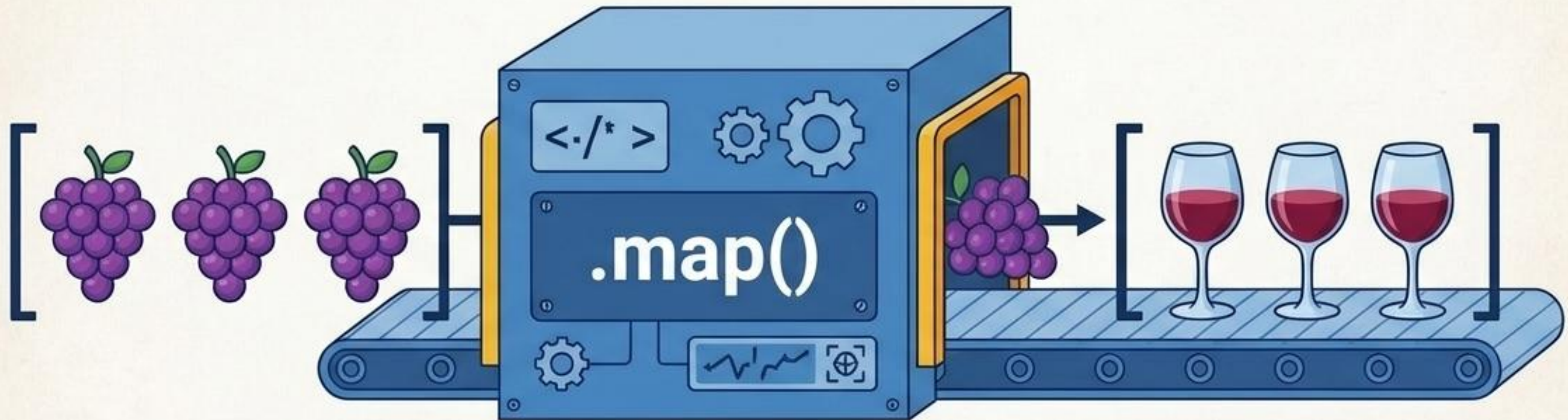


Solução:

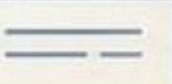
A propriedade **innerHTML** é a ponte que nos permite "despejar" strings de código HTML diretamente dentro de uma **** ou **<div>**.

A Máquina de Transformação: .map()

Passa por **cada item** da lista, aplica uma transformação, e devolve um novo array com exatamente o **mesmo tamanho**.

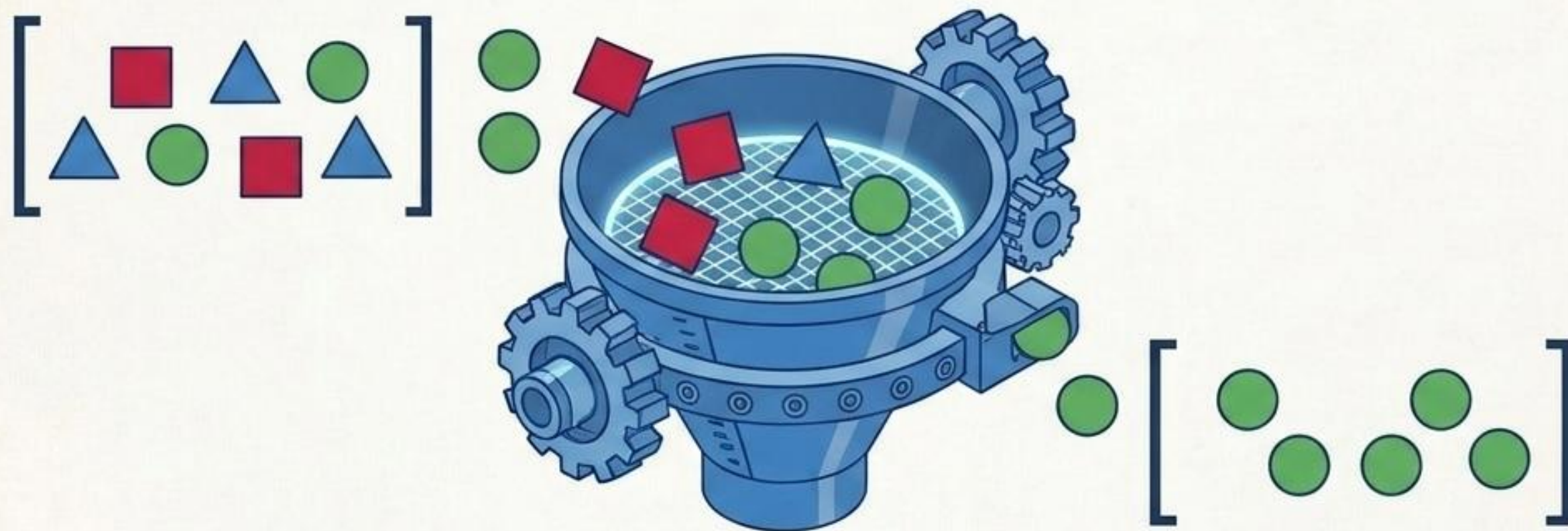


```
const descricoes = tarefas.map((tarefa) => {  
  return tarefa.descricao;  
});
```

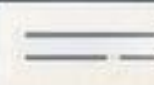
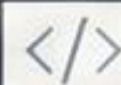


🔴 A Peneira de Condições: `.filter()`

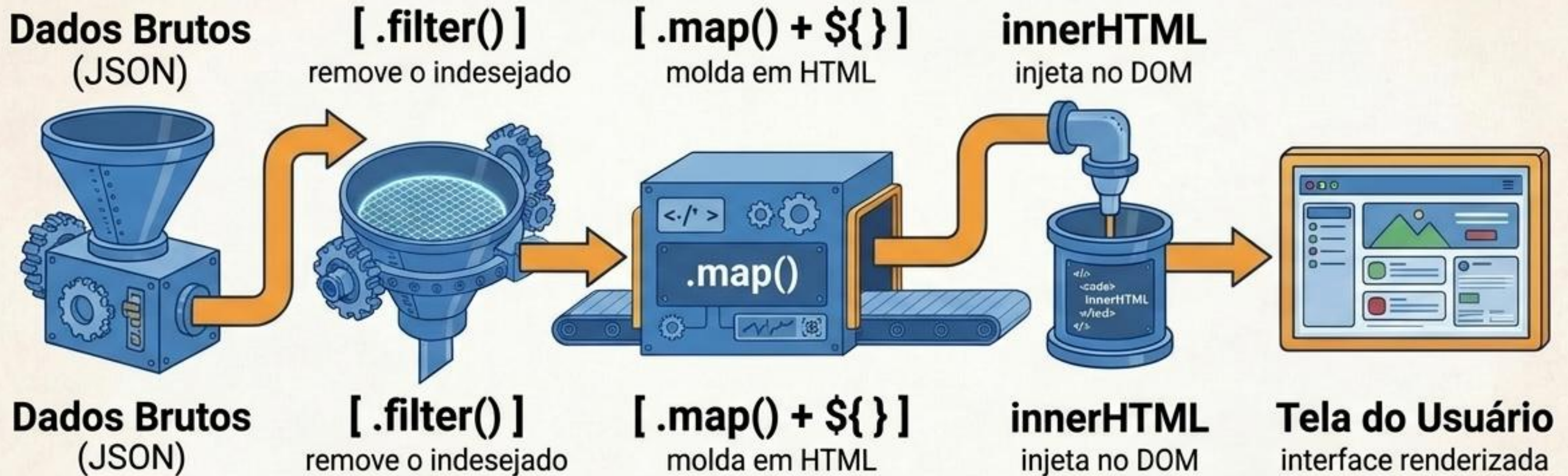
Testa cada **item** com uma condição. Devolve um novo array apenas com os itens que **passaram no teste** (onde a condição é verdadeira).



```
const penderes = tarefas.filter((tarefa) => {  
  return tarefa.concluida === false;  
});
```



Síntese: O Pipeline de Dados Moderno



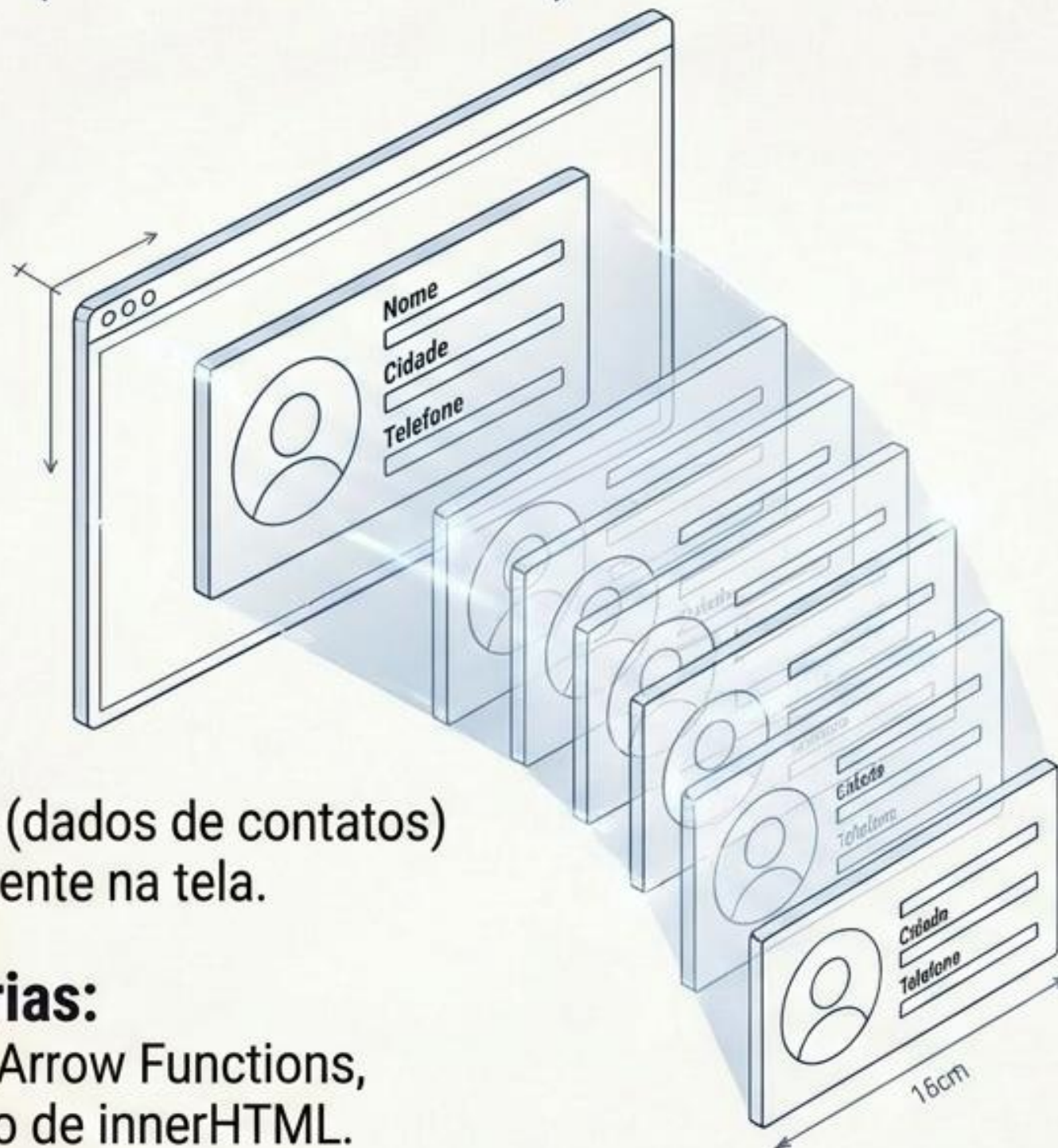
É exatamente assim que grandes plataformas como **Instagram, Netflix e Twitter** geram conteúdo na tela de forma dinâmica e instantânea!

O Seu Momento de Codificar

Chegou a **hora de plugar as ferramentas elétricas na tomada**. Teremos dois desafios progressivos de manipulação de dados e renderização no DOM.

O instrutor assumirá agora o papel de 'Gerente de Projeto' para guiar a sua execução.

Desafio 2 (Individual): Lista de Contatos

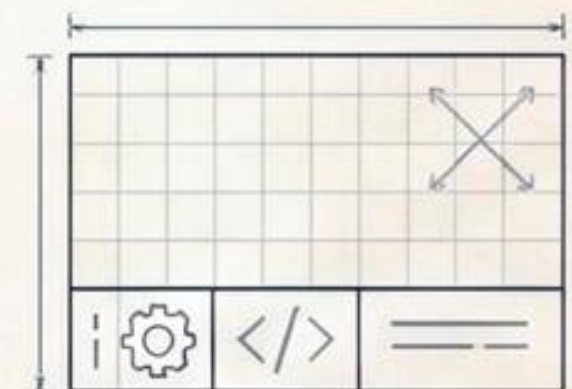


Objetivo:

Pegar um array de objetos (dados de contatos) e renderizá-los dinamicamente na tela.

Ferramentas Necessárias:

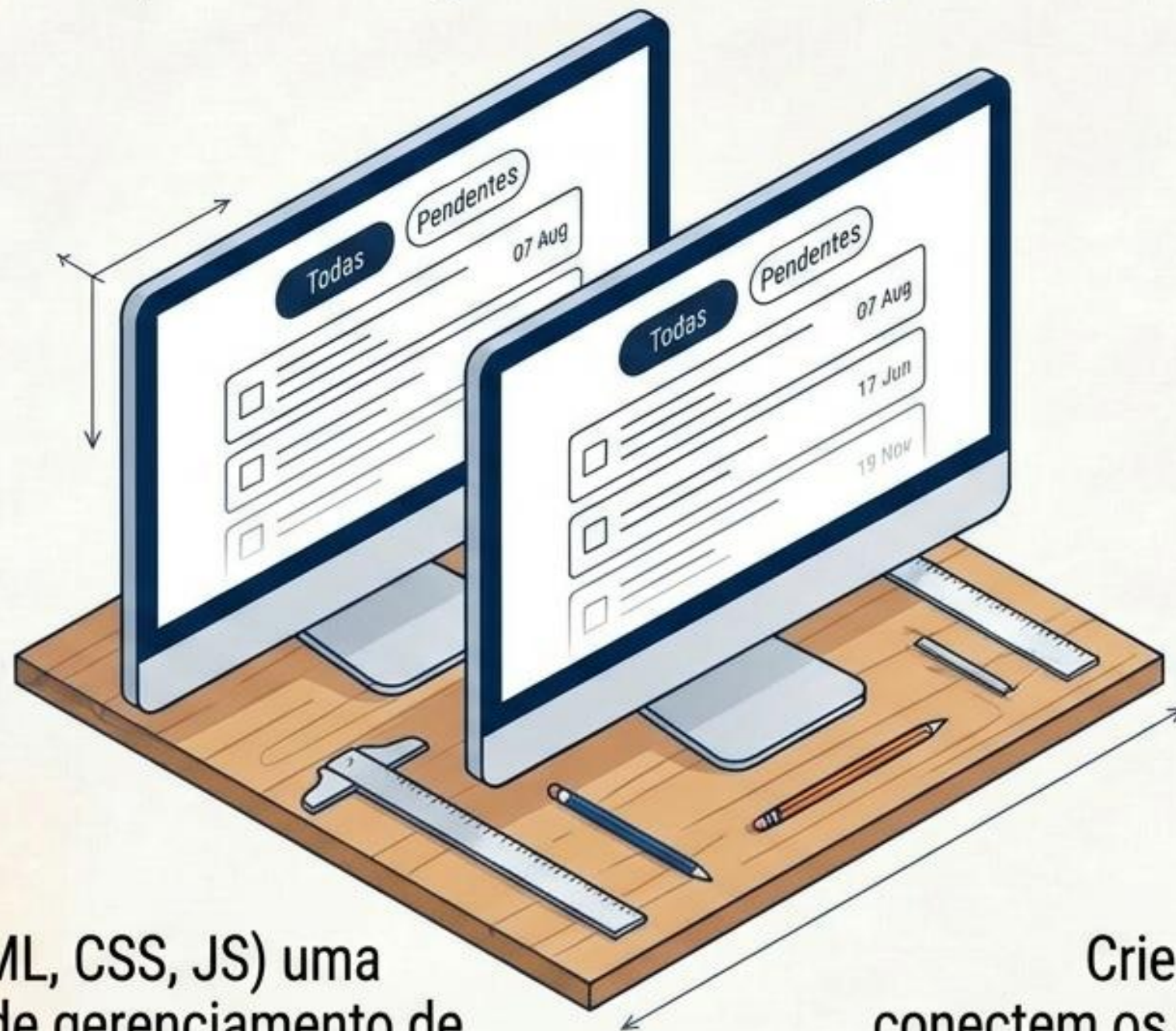
Laço **for...of** (ou **.map()**), Arrow Functions, Template Literals, e injeção de innerHTML.



Requisitos da Entrega

- [] Utilizar o arquivo script.js base fornecido na atividade.
- [] Criar uma função auxiliar utilizando a sintaxe de **Arrow Function** =>.
- [] Interpolar os dados (nome, cidade, telefone, foto) na estrutura HTML utilizando **Template Literals** `\${}`.
- [] Injetar a string resultante na **** principal da página.

Desafio 3 (Em Duplas): Projeto Taskflow



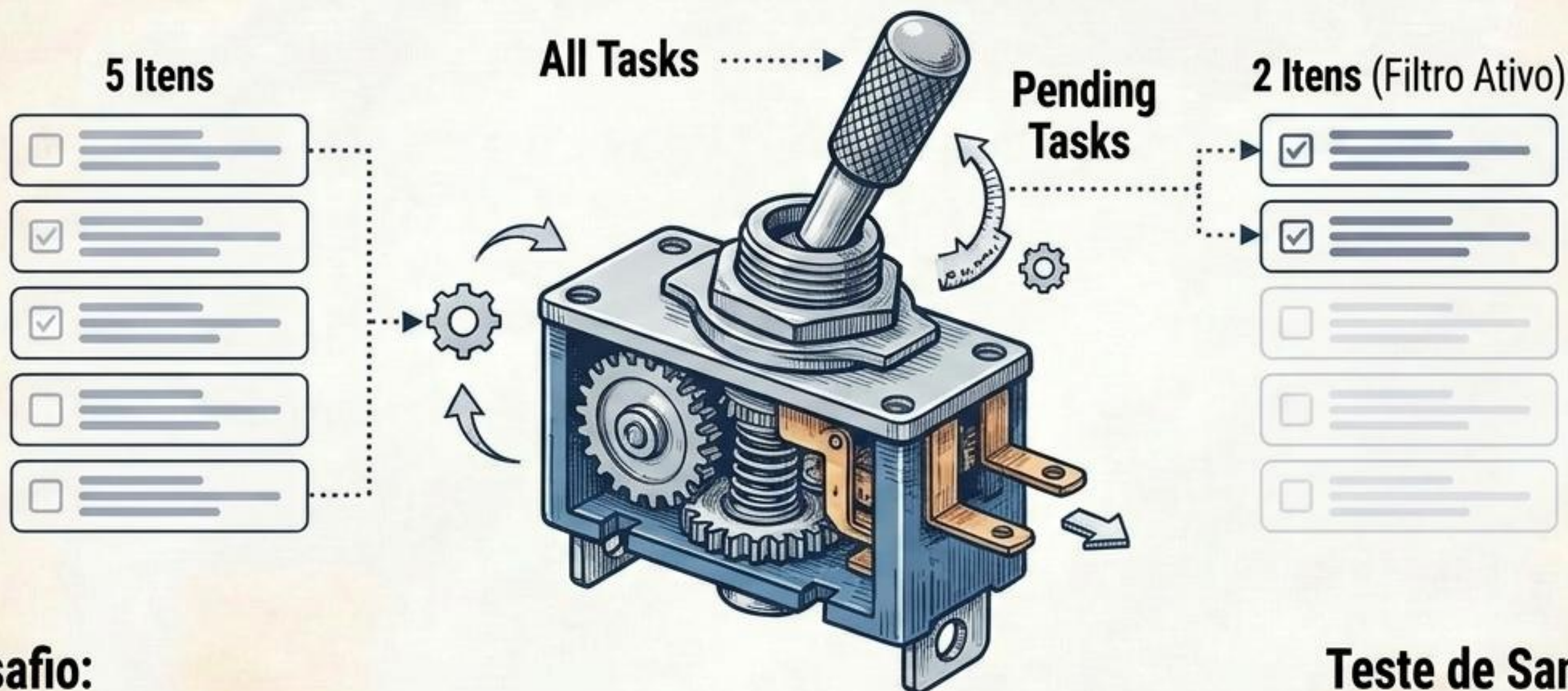
Objetivo:

Construir do zero (HTML, CSS, JS) uma interface minimalista de gerenciamento de tarefas com filtragem dinâmica de dados.

Setup Inicial:

Criem a estrutura visual e conectem os seus scripts antes de iniciar a lógica de dados.

Dominando a Lógica de Filtragem



O Desafio:

Utilizar o método `.filter()` para criar um novo array que contenha apenas as tarefas que possuem a propriedade `concluida: false`.

Teste de Sanidade:

Antes de enviar para o HTML, exiba o array resultante no `console.log()` para garantir que a sua 'peneira' funcionou corretamente.

Bônus da Dupla: Re-renderização Interativa

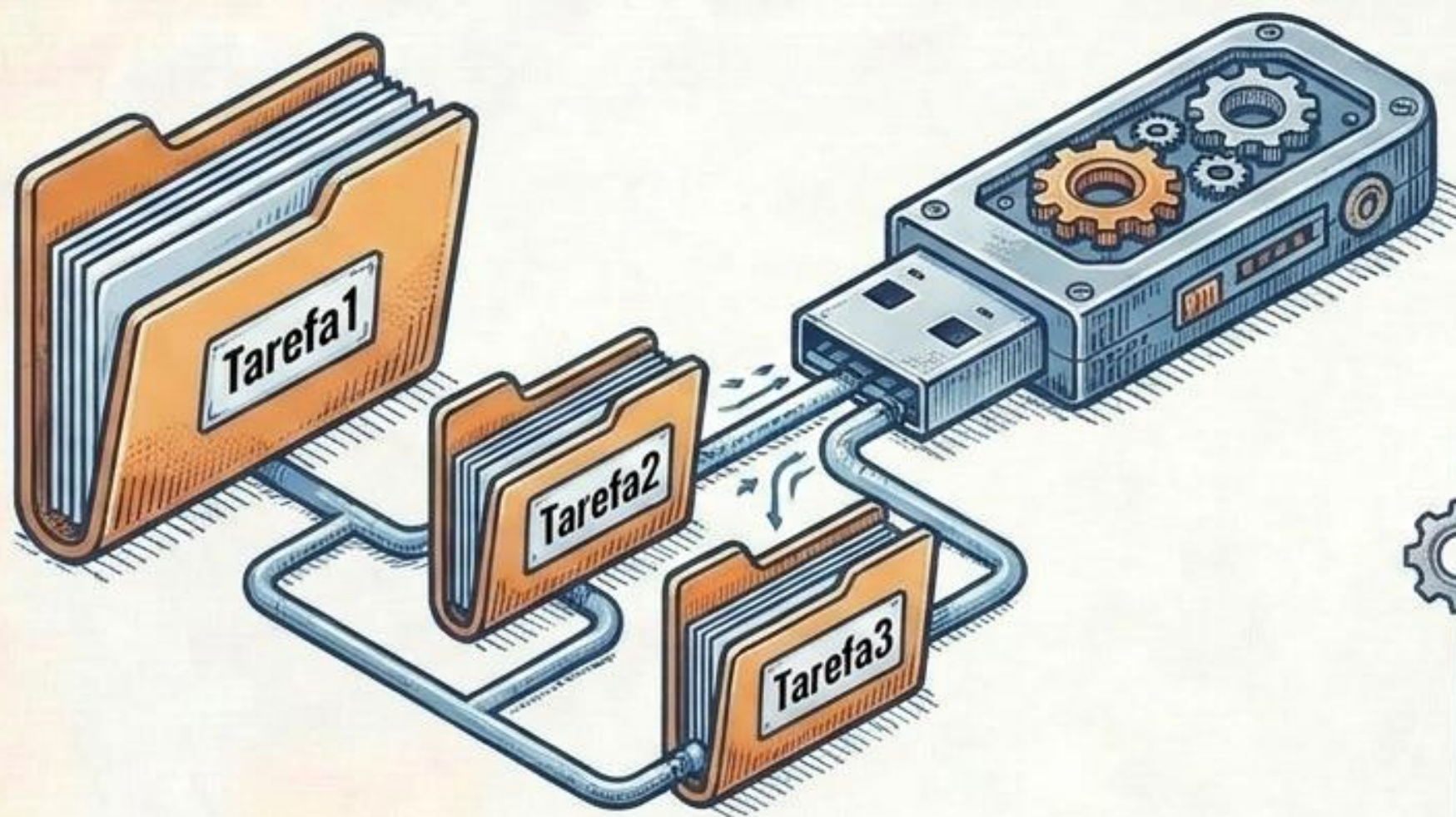


Missão:

Conectar dois botões no HTML: 'Mostrar Todas' e 'Mostrar Pendentes'.

Ação: Ao clicar nestes botões, a lista na tela deve ser limpa e re-renderizada exibindo apenas os dados correspondentes. (Dica: Esta é a essência de como o React funciona!)

O Protocolo de Entrega



- + **Estrutura:** Organizem os arquivos rigorosamente em pastas separadas.
- **Entrega Final:** A entrega será feita via pen-drive ao final do tempo regulamentar do laboratório.
- ±

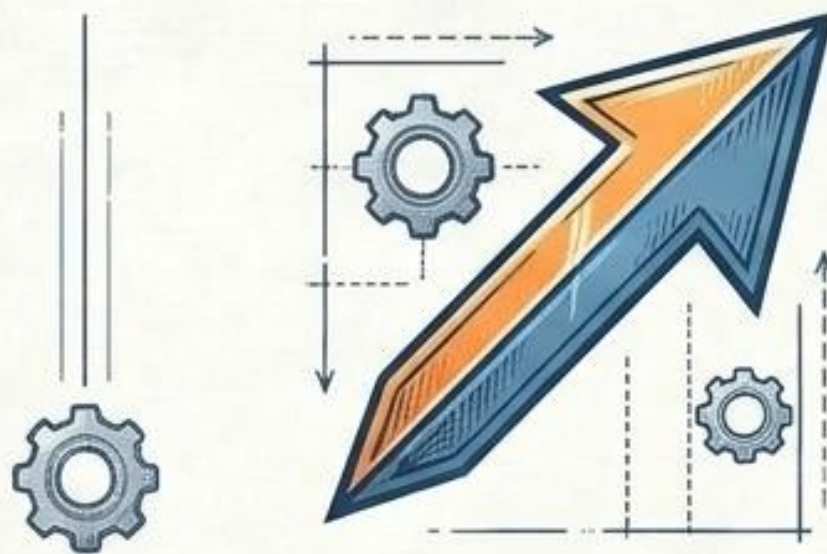
Regra de Ouro:

O instrutor atuará como Tech Lead. Tirem dúvidas sobre as regras de negócio e bloqueios técnicos, mas o código final é responsabilidade da dupla!

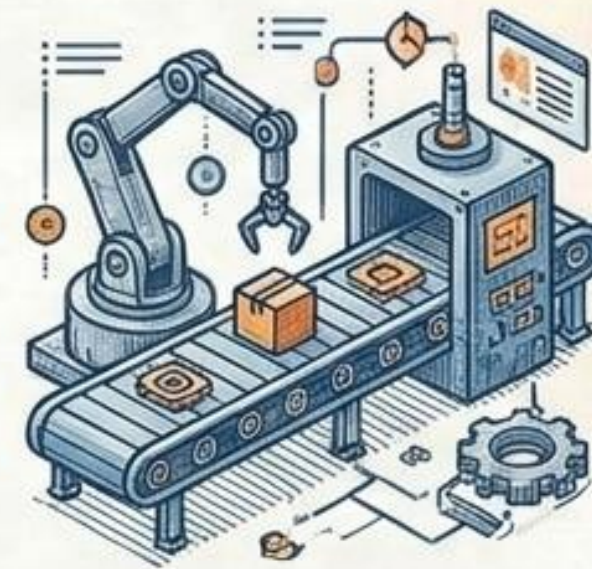
O Seu Novo Arsenal Front-end



Variáveis Seguras:
Substituímos o velho var
pela previsibilidade do let
e const.



Sintaxe Limpa:
Adotamos Arrow Functions
e Template Literals para um
código direto.



Manipulação Inteligente:
Descobrimos como processar
grandes volumes de dados
usando `.map()` e `.filter()`.

**Preparem-se, pois estas ferramentas elétricas são a
fundação absoluta para a introdução ao React!**